Python Notes (Lecture 5)

Loops

1. Loops in Python

Loops are used to **repeat instructions**. Python has two kinds of loops: `while` and `for`.

---

2. while Loop

A `while` loop keeps running **as long as its condition is True**.

```
while condition:
    # some work
```

Print "hello" 5 times:

```
count = 1
while count <= 5:
    print("hello")
    count += 1
```

Output:

```
hello
hello
hello
hello
hello
```

Print numbers from 1 to 5:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

Output:

```
1
2
3
4
5
```

The variable `i` is called the **iterator**: it changes in every round and finally makes the condition False.

Infinite loop: if the iterator is never updated, the condition stays True forever and the loop never ends.

```
i = 1
while i <= 5:
    print(i)     # i is never increased: runs forever!
```

In VS Code or a terminal, press **Ctrl + C** to stop an infinite loop.

Let's Practice : while

Q1. Print numbers from 1 to 100.

```
i = 1
while i <= 100:
    print(i)
    i += 1
```

Output:

```
1
2
3
...
100
```

Q2. Print numbers from 100 to 1.

```
i = 100
while i >= 1:
    print(i)
    i -= 1
```

Output:

```
100
99
98
...
1
```

Q3. Print the multiplication table of a number n.

```
n = int(input("Enter a number: "))

i = 1
while i <= 10:
    print(n, "x", i, "=", n * i)
    i += 1
```

Output:

```
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
...
5 x 10 = 50
```

Q4. Print the elements of the following list using a loop.

`[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]`

```
nums = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

idx = 0
while idx < len(nums):
    print(nums[idx])
    idx += 1
```

Output:

```
1
4
9
...
100
```

Q5. Search for a number x in this tuple using a loop.

`(1, 4, 9, 16, 25, 36, 49, 64, 81, 100)`

```
nums = (1, 4, 9, 16, 25, 36, 49, 64, 81, 100)
x = 36

idx = 0
while idx < len(nums):
    if nums[idx] == x:
        print("Found at index", idx)
    idx += 1
```

Output:

```
Found at index 5
```

---

3. break and continue

- **break**: terminates the loop immediately when encountered.
- **continue**: skips the rest of the current iteration and moves on to the next one.

break: stop searching as soon as the number is found.

```
nums = (1, 4, 9, 16, 25, 36, 49, 64, 81, 100)
x = 36

idx = 0
while idx < len(nums):
    if nums[idx] == x:
        print("Found at index", idx)
        break               # no need to check further
    idx += 1

print("End of search")
```

Output:

```
Found at index 5
End of search
```

continue: print numbers from 1 to 10, but skip multiples of 3.

```
i = 0
while i < 10:
    i += 1
    if i % 3 == 0:
        continue            # skip this number
    print(i)
```

Output:

```
1
2
4
5
7
8
10
```

Note: in a `while` loop, update the iterator **before** `continue`, otherwise the loop gets stuck.

---

4. for Loop

A `for` loop is used for **sequential traversal**: going through each item of a list, string, tuple, etc.

```
for el in sequence:
    # some work
```

```
nums = [1, 2, 3]
for el in nums:
    print(el)
```

Output:

```
1
2
3
```

```python
for ch in "Python":
    print(ch)
```

Output:

```
P
y
t
h
o
n
```

for Loop with else

The `else` block runs when the loop finishes normally. It does **not** run if the loop is stopped by `break`.

```python
nums = [1, 2, 3]
for el in nums:
    print(el)
else:
    print("END")
```

Output:

```
1
2
3
END
```

```python
nums = [1, 2, 3]
for el in nums:
    if el == 2:
        break
    print(el)
else:
    print("END")    # skipped because of break
```

Output:

```
1
```

Let's Practice : for

Q1. Print the elements of the following list using a loop.

`[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]`

```
nums = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
for num in nums:
    print(num)
```

Output:

```
1
4
9
...
100
```

Q2. Search for a number x in this tuple using a loop.

`(1, 4, 9, 16, 25, 36, 49, 64, 81, 100)`

```
nums = (1, 4, 9, 16, 25, 36, 49, 64, 81, 100)
x = 49

idx = 0
for num in nums:
    if num == x:
        print("Found at index", idx)
        break
    idx += 1
else:
    print("Not found")
```

Output:

```
Found at index 6
```

---

5. range()

`range()` returns a sequence of numbers. It starts from **0** by default, increases by **1** by default, and stops **before** the given stop number.

```
range(start, stop, step)    # start and step are optional
```

```
for el in range(5):          # 0 to 4
    print(el)
```

Output:

```
0
1
2
3
4
```

```
for el in range(1, 5):       # 1 to 4
    print(el)
```

Output:

```
1
2
3
4
```

```
for el in range(1, 10, 2):   # 1 to 9, step of 2
    print(el)
```

Output:

```
1
3
5
7
9
```

Let's Practice : for and range()

Q1. Print numbers from 1 to 100.

```
for i in range(1, 101):
    print(i)
```

Output:

```
1
2
3
...
100
```

Q2. Print numbers from 100 to 1.

```
for i in range(100, 0, -1):    # negative step counts down
    print(i)
```

Output:

```
100
99
98
...
1
```

Q3. Print the multiplication table of a number n.

```
n = int(input("Enter a number: "))

for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

Output:

```
Enter a number: 7
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

---

6. pass Statement

`pass` is a **null statement** that does nothing. It is used as a placeholder for code you will write later, because Python does not allow an empty block.

```
for el in range(10):
    pass        # to be written later

print("Program continues")
```

Output:

```
Program continues
```

It is also commonly used in exception handling, which we will learn later.

Let's Practice

Q1. Find the sum of the first n numbers (using while).

```
n = int(input("Enter n: "))

total = 0
i = 1
while i <= n:
    total += i
    i += 1

print("Sum =", total)
```

Output:

```
Enter n: 5
Sum = 15
```

Q2. Find the factorial of n (using for).

```
n = int(input("Enter n: "))

fact = 1
for i in range(1, n + 1):
    fact *= i

print("Factorial =", fact)
```

Output:

```
Enter n: 5
Factorial = 120
```

Factorial of 5 = 1 × 2 × 3 × 4 × 5 = 120.

APNA COLLEGE